

Game of Life: Parallel & Distributed System

Zehao (Dylan) Lin, Yiyang (Tommy) Dong, Zhitong Liu
School of Computer Science, Faculty of Engineering
University of Bristol, Bristol, UK
{nj23727, of23238, wr23070}@bristol.ac.uk

I. INTRODUCTION

In this project, we enhanced the performance of the Game of Life by developing multiple implementations based on the same skeleton code: a parallel version using channels, a distributed version on AWS, and further extensions including **Halo Exchange**, **Parallel Distributed System**, and an **SDL Live View** in distributed system.

II. PARALLEL IMPLEMENTATION - FUNCTIONALITY AND DESIGN

A. Overview of Implementation

The program simulates the Game of Life using standard rules, evolving the grid over specified turns, supporting customizable grid sizes, and handling user inputs for pausing, saving, and quitting.

B. Goroutines Used and Their Interactions

The program utilises several goroutines to achieve concurrency, each with specific responsibilities and communication patterns:

- **Distributor Goroutine:** Manages simulation flow, divides the grid among workers, handles user inputs (such as pausing, saving, or quitting), and communicates with IO and event goroutines via `distributorChannels`, which includes channels for events, IO commands, filenames, output data, and key presses.
- **Worker Goroutines:** Compute the next state of grid portions concurrently. The number of workers is specified by `p.Threads`, allowing scalability. Workers process assigned sections and send computed segments back to the distributor goroutine via `tempWorld` channels for merging into the global grid.
- **IO Goroutine:** Handles all input/output operations, reading and writing grid states from/to PGM files upon initiation or user request, using `ioChannels` to receive commands and filenames from the distributor and send or receive grid data. This separation ensures IO does not block the main logic.
- **Event Goroutine:** Responsible for communicating events such as cell state changes, turn completions, and user commands to the GUI or testing framework. It uses the `events` channel to share information. Events are sent asynchronously, allowing the simulation to continue running without waiting for the event handling to complete.

These goroutines interact primarily via channels, ensuring safe communication and synchronization without the need for explicit locks or shared memory. The use of structured channels (such as `distributorChannels` and `ioChannels`) encapsulates related communication channels, enhancing code organization and readability.

C. Problems Solved and Solutions

During the development of the parallel Game of Life implementation, we encountered several key challenges that needed to be addressed to ensure efficient and effective performance. This section outlines these challenges and the solutions we implemented.

1) *Efficient Parallel Computation:* Maximizing the use of multiple CPU cores to reduce the overall computation time per simulation turn was essential. The grid was partitioned horizontally, assigning contiguous rows to each worker goroutine based on the number of threads specified by `p.Threads`.

This approach ensures that each worker has a balanced workload, minimising idle time and preventing bottlenecks. By processing different sections of the grid concurrently, we effectively reduce the computation time per turn. However, running on an M1 Max MacBook Pro with 10 cores showed that performance gains plateau beyond 10 threads. We'll discuss more about this in the Testing and Critical Analysis section.

2) *Synchronization and Communication:* To ensuring safe and efficient communication between multiple goroutines without introducing significant synchronization overhead. The implementation utilizes structured channels encapsulated within `distributorChannels` and `ioChannels` to manage different aspects of the simulation.

For instance, `tempWorld` channels collect the computed grid sections from workers for merging, while the `events` channel handles asynchronous event notifications such as cell state changes and turn completions. User inputs are managed through the `keyPresses` channel, allowing commands like pause, save, or quit to be processed without blocking the main simulation loop.

3) *Edge Wrapping and Boundary Conditions:* Implementing toroidal (wrap-around) boundary conditions efficiently was necessary to ensure that the simulation behaves correctly at the edges of the grid. We optimized `countLiveNeighbors` using modular arithmetic for accurate edge wrapping, ensuring the grid behaves as a torus. Precomputing neighbor offsets

and streamlining wrapping calculations further reduced the computational overhead, allowing edge cells to be processed with the same efficiency as interior cells.

4) *Optimizing Memory Usage:* To managing memory efficiently, we particularly with large grid sizes and multiple goroutines. Excessive memory allocation and copying can lead to increased latency and reduced performance. We avoid excessive memory allocation and copying by reusing memory buffers and alternating between two grid states (current and next). Specifically, after computing the next state of a segment of the grid, the results are merged into a new grid (mergeWorld), which replaces the current grid (world).

Additionally, we carefully manage the goroutine and channel life-cycles ensures that memory is released promptly when no longer needed, preventing memory leaks and optimizing resource utilization.

5) *Handling User Inputs and Control Commands:* To provide responsive control over the simulation—such as pausing, saving, or quitting—we implemented asynchronous handling of user inputs without disrupting ongoing computations.

The distributor goroutine monitors the `keyPresses` channel using non-blocking `select` statements and responds accordingly. Commands like 's' (save), 'q' (quit), and 'p' (pause) are processed asynchronously, allowing the simulation to remain responsive.

Upon receiving a save command, the distributor calls the `outputImage` function to write the current grid state to a PGM file. Pause and quit commands alter the simulation state and ensure that necessary cleanup operations are performed gracefully. This design seamlessly integrates user interactions into the simulation loop, providing a smooth and responsive user experience.

D. Extension - Memory Sharing

In this implementation, we have redesigned the parallel structure by eliminating the use of Go channels in favor of traditional synchronization mechanisms such as **mutexes** and **condition variables**. This design maintains the separation of concerns by keeping workers, IO, SDL, and distributor as separate goroutines while ensuring thread-safe communication between them.

The `sdystate` and `shareIOState` structures manage synchronization for SDL and IO operations by using mutexes and conditional variables respectively. Mutexes guarantees that only one goroutine can access or modify a shared resource at a time, ensuring data integrity. While condition variables help coordinate interactions between goroutines without the need for busy-wait loops. By applying `Signal()` and `Wait()` for inter-goroutine signaling, we achieve more explicit control over state management and synchronization points.

III. PARALLEL IMPLEMENTATION - TESTING AND CRITICAL ANALYSIS

A. Description of Experiments and Analysis

To evaluate the performance improvements resulting from parallelization in our *Game of Life* implementation, we con-

ducted a series of benchmarks varying the number of worker goroutines. The benchmarks were designed to measure the total execution time for simulating a fixed number of turns on a grid of a specific size. All experiments were performed on an Apple M1 Max MacBook Pro with a 10-core CPU, ensuring consistency in the hardware environment.

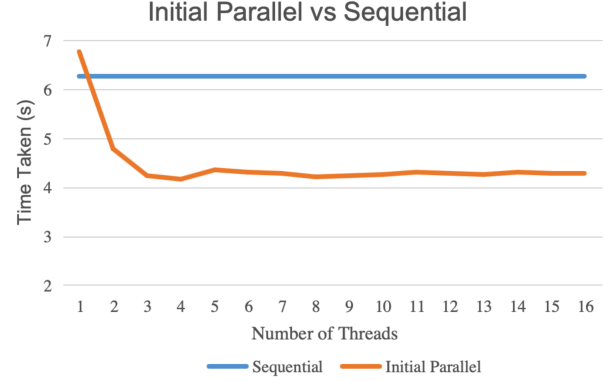


Fig. 1. Execution Time vs. Number of Threads for Initial Parallel and Sequential Implementations.

1) *Sequential vs. Initial Parallel Implementation:* Figure 1 compares the execution time of the sequential implementation with the initial parallel implementation using goroutine workers. The sequential execution time remains constant at approximately 6.3 seconds, as it does not utilize concurrency.

In the initial parallel implementation, as the number of threads increased, the time taken decreased. With 16 threads, the implementation was **1.42x** faster than the sequential implementation with 1 thread. The perfect improvement would be a runtime that halves as the number of threads is doubled. This is not the case here, because we measured the total time taken to input an image, apply the game logic and output an image. The workers apply the game logic in parallel, but the image input and output, which happens outside the workers, is not parallelisable.

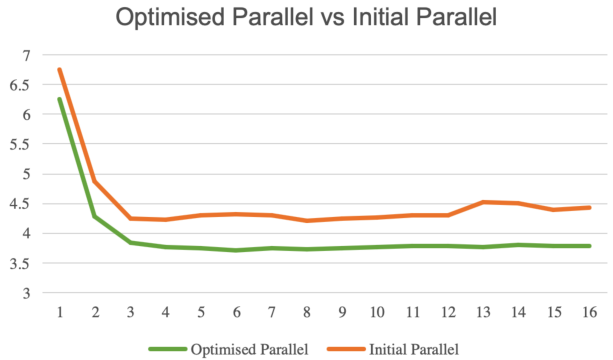


Fig. 2. Execution Time vs. Number of Threads for Optimized and Initial Parallel Implementations. The optimized version reduced execution time from 6.7 seconds to 6.4 seconds with 1 thread, and from 4.87 seconds to 4.27 seconds with 2 threads.

2) *Optimized Parallel Implementation*: Figure 2 presents a comparison between the optimized parallel implementation and the initial parallel version. In the optimized version, we improved the algorithm by removing redundant operations. Specifically, we eliminated the unnecessary use of an `immutableWorld` function, which was initially used to prevent race conditions by passing an immutable copy of the world to the `calculateNextTurn` function. However, since `calculateNextTurn` already operates on a copied version of the world, this step was redundant.

As a result, the execution time improved across all thread counts. The optimized implementation was approximately **1.14x** faster than the initial parallel version. This demonstrates that optimizing code logic can significantly impact performance, in addition to parallelization.

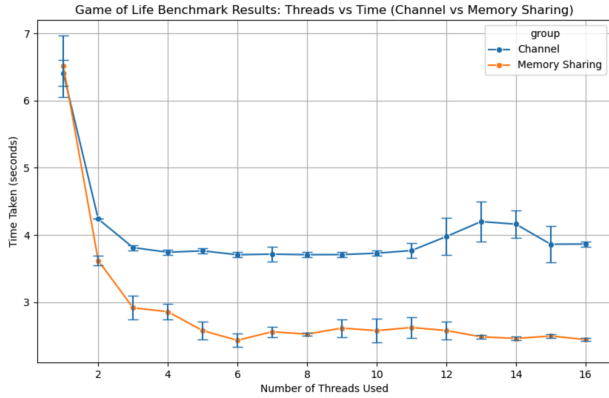


Fig. 3. Final Parallel Implementation with Error Bars vs. Memory Sharing Implementation with Error Bars

3) *Final Parallel Implementation with Error Bars*: Figure 3 shows the execution time of our final parallel implementation, with error bars representing the variability across the six runs for each thread count. As the number of threads increases from 1 to 10 (the number of physical CPU cores), the execution time decreases significantly, dropping from approximately 6.4 seconds with 1 thread to around 3.7 seconds with 10 threads, achieving a speedup of nearly **1.73x**.

Beyond 10 threads, the performance gains begin to plateau, and the error bars become larger. This is due to the overhead associated with managing more threads than physical cores, leading to increased context switching and synchronization costs. At 16 threads, the execution time may even increase slightly due to these overheads.

Figure 3 also shows the execution times of our memory sharing implementation compared to the channel version, with error bars representing the variability across the runs. The Memory sharing implementation demonstrates a significant reduction in execution time, decreasing from approximately 4.0 seconds in the channel version to around 2.8 seconds in the memory sharing version. This results in a speedup of nearly **1.4x**, reflecting a 30% decrease in execution time.

The performance improvement is attributed to the use of the Mutexes and condition variables which enhances resource

utilization and reduces overhead compared to the channel version. The memory sharing implementation shows smaller error bars, indicating more consistent performance and stability across multiple runs. These findings underscore the effectiveness of the Mutex approach in achieving faster execution times and improved reliability, demonstrating its advantage over the channel implementation in this context.

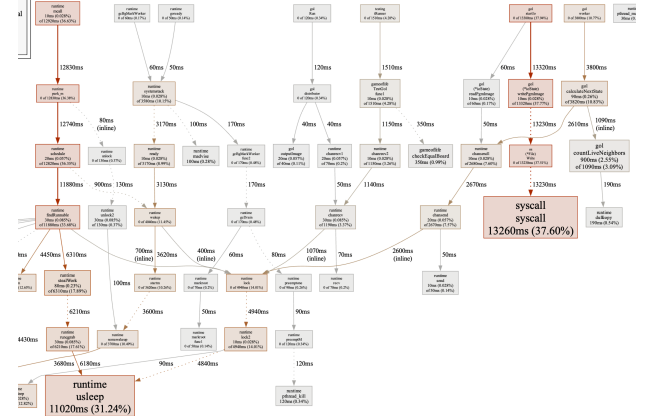


Fig. 4. CPU Profile of the Final Parallel Implementation

4) *CPU Profiling Analysis*: Figure 4 presents the CPU profile generated using Go's `pprof` tool, providing a detailed breakdown of CPU time consumed by various functions in the program. The profile reveals that a significant portion of the execution time, 37.60%, is consumed by system calls (`syscall`), primarily due to frequent file write operations in `writePgmImage()`. These serialized I/O operations are blocking and introduce high context switching overhead, limiting the efficiency of parallel processing. Additionally, 31.24% of the runtime is attributed to `runtime.usleep`, likely resulting from goroutines waiting for I/O completion in `readPgmImage()`. This indicates that I/O operations are a major bottleneck in our implementation.

B. Analysis of Factors Affecting Performance

The performance of our parallel implementation is influenced by several key factors contributing to both its strengths and limitations.

1) *System and Hardware Limitations*: CPU profiling indicates that I/O operations are a significant bottleneck. The blocking nature of `readPgmImage()` and `writePgmImage()` leads to substantial time spent in system calls and goroutine sleep states. Since these I/O tasks are serialized and cannot be parallelized, they limit overall speedup.

Additionally, the M1 Max CPU has 10 physical cores. While it supports more threads through hyper-threading, performance gains are limited beyond the physical core count due to context switching overhead and resource contention. Memory bandwidth and cache sizes also constrain scalability; increased concurrency can lead to contention for these resources, affecting overall performance.

2) *Algorithmic Efficiency and Load Balancing*: The improvements observed in the optimized implementation highlight the importance of algorithmic efficiency. By eliminating redundant operations and unnecessary data copying, we reduced execution time across all thread counts. Efficient algorithms minimize computational overhead and make better use of hardware resources, complementing the benefits gained from parallelization.

However, we partitioned the grid evenly among worker goroutines to ensure balanced workloads, but if the distribution of live cells is uneven, some workers may have more computational work than others, leading to inefficiencies. Implementing dynamic load balancing could allow flexible task allocation based on actual workload, potentially improving CPU utilization and overall performance.

C. Conclusion

In summary, while parallelization significantly improves performance up to the number of physical CPU cores, factors such as I/O bottlenecks, communication overhead, load imbalance, synchronization costs, and hardware constraints limit scalability. Addressing these factors—through optimizing I/O operations, implementing dynamic load balancing and refining algorithms—can further enhance the performance of the parallel implementation.

IV. DISTRIBUTED IMPLEMENTATION - FUNCTIONALITY AND DESIGN

A. Overview of Implementation

The distributed version of the Game of Life is designed to be faster and more scalable by using multiple AWS nodes and Remote Procedure Call (RPC) communication. In simpler terms, the goal is to split up the work across several servers so that each one handles a smaller piece of the game. This way, it runs faster than if we tried to do everything on just one machine.

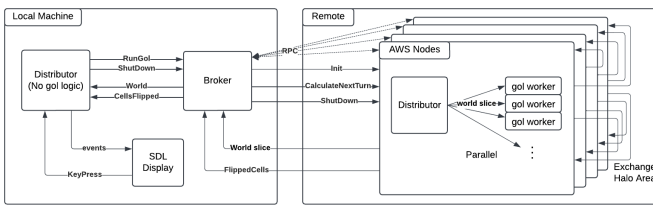


Fig. 5. The diagram of System Design

B. Component Design

The system is set up in a way that spreads out the workload across multiple AWS nodes. There are three main parts to this system: the **distributor**, **broker**, and **remote controller (AWS Node)**. Each plays a different role in making sure the game runs smoothly:

- **Distributor**: The Distributor runs on the local machine and is responsible for handling user input, managing the game flow. As shown in the diagram above, it connects to

the SDL Display for keyboard events and state updates. This enabling a **SDL live view** in addition to the Broker implementation (step 1-6). Distributor communicates with the Broker via RPC to send the initial game state and receive updates.

- **Broker**: Acting as the central coordinator, the Broker manages the distribution of computational tasks to the Worker Nodes. It partitions the game board into segments, assigns these segments to different Workers, i.e. *AWS Nodes*, and aggregates the results after each iteration. The Broker uses RPC to communicate with both the Controller and the Worker Nodes, abstracting the complexity of network communication and providing a centralized point for task scheduling and result collection.
- **Worker Nodes**: Deployed on AWS instances, the Worker Nodes perform the core computation of the Game of Life. Each Worker receives a segment of the game board from the Broker, computes the next state for its assigned segment, and returns the updated segment back to the Broker. Internally, we implemented **Parallel Distributed System**, so that each Worker Node can utilize multiple goroutines to parallelize computation within the node, maximizing hardware utilization.

C. System Design

The choice of using RPC for communication was driven by its simplicity and efficiency for remote method invocation in a distributed system. The modular design with a Broker facilitates scalability and fault tolerance, as new Worker Nodes can be added or removed with minimal changes to the system.

1) *Data Transmission and Halo Exchange Mechanism*: In each iteration, the Local Distributor sends the initial game state to the Broker, which then partitions the board and distributes segments to the Worker Nodes. In the initial setup, halo data synchronization between nodes was handled through central broker, requiring each iteration to pass data through this central point. This design led to noticeable communication overhead, which impacted system performance. To address this, we implemented a **Halo Exchange** mechanism.

Instead of having AWS Nodes synchronize boundary data through the Broker, neighboring nodes communicate directly via RPC to exchange their halo regions. This peer-to-peer communication reduces the reliance on the central Broker and decreases latency, as illustrated in Figure 5.

2) *Fault Tolerance*: We design a mechanism that detects and handles workers' node failure dynamically. Our **Broker** continuously monitors each worker node's status **every 5 seconds**. If a worker fails to respond, the Broker immediately removes it from the active list and reallocates its tasks to other active nodes, ensuring the computation continues smoothly.

D. Problems Solved and Solutions

During the development of the distributed Game of Life implementation, we encountered several key challenges that needed to be addressed to ensure efficient, scalable, and responsive performance.

1) *Scaling Computation Across Multiple Nodes:* Scaling the computation across multiple AWS nodes introduced complexities in workload distribution, data consistency, and inter-node communication. A significant challenge was partitioning the Game of Life grid among nodes while ensuring that boundary cells correctly interact with neighboring nodes.

Solution: We introduced a **Broker** component to manage task distribution and act as an intermediary between the local controller and the worker nodes. The Broker partitions the game board into slices and assigns each slice to a worker node. Each worker receives its assigned slice along with necessary halo regions (neighboring rows) to accurately compute cell states at the boundaries.

2) *Efficient Communication and Data Consistency:* Managing communication between the local controller, Broker, and worker nodes was critical, especially given the need for efficient data transfer and maintaining data consistency across distributed components. Initially, we passed the entire subworld to each worker, and within each worker, the subworld was copied to avoid race conditions. However, this approach was inefficient and led to significant delays, particularly with larger grids.

Solution: To optimize data transfer and improve efficiency, we redesigned our communication algorithm. Instead of passing the entire subworld, each worker node only computes the cells that need to be flipped (i.e., cells whose state changes in the next turn). The workers then send back to the Broker only the list of these cells, rather than the entire subworld. The Broker updates the global world state by flipping these cells and then passes the updated world back to the controller. We implemented the following strategies:

- **Reduced Data Transmission:** By transmitting only the coordinates of cells that have changed state, we significantly reduced the amount of data sent over the network, minimizing network load and latency.
- **Optimized Data Structures:** Developed efficient data structures to store and transmit the list of flipped cells, including their coordinates and new states.
- **Efficient Data Handling:** The Broker efficiently updates the global world state by flipping the states of the received cells, ensuring data consistency across the distributed system.

Data consistency was maintained by centralizing the updates at the Broker. By only transmitting the necessary changes rather than entire slices, we preserved accuracy in cell state updates while greatly improving communication efficiency. This approach allowed us to overcome the performance bottlenecks of our initial implementation and achieve a more scalable and responsive system.

V. DISTRIBUTED IMPLEMENTATION - TESTING AND CRITICAL ANALYSIS

A. Description of Experiments and Analysis

1) *Methodology:* We evaluated the performance of our distributed implementation by conducting benchmark tests

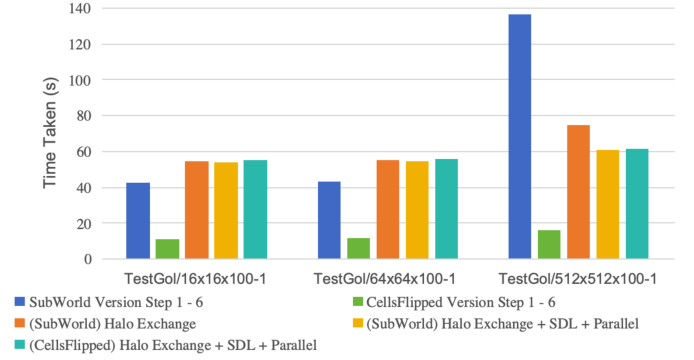


Fig. 6. Performance Comparison of Different Distributed System

under various configurations. We deployed multiple AWS `t2.micro` instances as Worker Nodes, processing different segments of the game board. The tests were performed on grid sizes of 16×16 , 64×64 , and 512×512 , each running for 100 iterations.

To ensure consistency and reliability, we ran multiple trials for each configuration and averaged the results. All experiments were conducted in a controlled environment using the same Apple M1 Max MacBook Pro. Network latency and bandwidth were considered in the analysis to account for communication overhead.

B. Results and Discussion

1) *Comparison between SubWorld and CellsFlipped Implementations:* In the SubWorld implementation, the entire world slice is sent to each AWS node, and within each node, the slice is copied to prevent race conditions. This design adds considerable communication overhead due to data replication, especially with large images (e.g. 512×512), which results in higher execution times.

On the other hand, the CellsFlipped setup improves data transfer by skipping internal replication within each node. Instead, it only sends the cells needing updates back to the broker, which integrates changes directly into the original world. This approach cuts down data transfer between nodes and reduces redundant calculations, enhancing communication and computation efficiency.

As shown at Figure 6, CellsFlipped version delivers a significant performance across all image sizes. For example, with the 512×512 image, the runtime of CellsFlipped is only 15.75 seconds, significantly faster than the 136.15 seconds of SubWorld, representing an **8.6x** improvement due to reduced data replication and lower communication overhead.

The implementation with Halo Exchange outperformed the basic (SubWorld) distributed system without Halo Exchange, especially on larger grids. For instance, on the 512×512 grid, the runtime decreased from 136 seconds to 74 seconds when using Halo Exchange. This shows that the runtime is **1.84x** faster.

The addition of SDL Live View and intra-node parallelism had a limited effect on performance, the runtime stabilizing around 60 seconds across all image sizes. This is because for the image size less than 64, this system is overkill since there are 4 AWS nodes and 16 worker threads in each of them. And for larger image, because `t2.micro` only have single-core CPU, and the largest we can use `t2.large` only have two cores CPU. As a result, the benefits of parallelism are diminished due to the limited number of physical cores, preventing the system from achieving its full potential.

2) *Impact of Halo Exchange and SDL Live View Extensions*: The introduction of the Halo Exchange mechanism in the distributed system significantly reduced the communication frequency and data volume between nodes, leading to a notable enhancement in overall system performance. In the SubWorld configuration, data exchanges among nodes occurred frequently, particularly for boundary data, which resulted in substantial communication overhead. This issue was particularly prominent with larger grid sizes, such as 512×512 . Without Halo Exchange, the runtime reached as high as 136 seconds, indicating that communication latency was severely affecting system performance. By implementing Halo Exchange, nodes only needed to share boundary information, greatly reducing the frequency and volume of data transmitted, which in turn brought the runtime down to 74 seconds. This considerable performance improvement demonstrates that Halo Exchange effectively mitigates communication overhead and enhances overall efficiency, especially for large-scale parallel computing tasks.

Additionally, we further extended the distributed system by incorporating SDL Live View and parallel processing to enable real-time observation of grid evolution during computation. This extension had a minimal impact on overall performance. As shown by the *yellow* and *teal bars* in Figure 6, the runtime for the 512×512 grid test, with SDL Live View and parallel processing, was similar to the system with only Halo Exchange, ranging between 60 and 74 seconds. Despite the added complexity of real-time visualization and extra parallelization, the overall runtime of system was not significantly affected, indicating that the optimization introduced by Halo Exchange had already alleviated most of the communication overhead, and the impact of SDL Live View was relatively minor.

C. Analysis of Factors Affecting Performance

1) *System and Hardware Limitations*: The performance of the distributed setup is heavily influenced by hardware capabilities and communication protocols, with primary bottlenecks arising from frequent boundary synchronization, especially on larger grids. Network latency and limited bandwidth between AWS nodes cause delays in halo exchanges, which are essential for maintaining grid consistency.

2) *Communication and Synchronization Overhead*: Worker goroutines use channels to communicate with the central controller, sending grid segments and updates. However, as the number of nodes and goroutines grows, the overhead from channel communication and synchronization also rises.

Each goroutine has to wait for data from neighboring nodes, causing delays, especially during synchronization at the end of each iteration. This delay significantly impacts performance for larger grids like 512×512 . To address this, asynchronous communication or pipelining could allow goroutines to keep processing without waiting for complete synchronization, reducing idle times and improving CPU use.

3) *Algorithmic Efficiency and Load Balancing*: While the halo exchange optimization reduces communication by sharing only boundary information, dividing the grid evenly among worker goroutines does not account for variations in computational load, especially when live cells are unevenly distributed. This imbalance can cause some nodes to be idle while others are overloaded, resulting in less efficient resource utilization.

Implementing dynamic load balancing based on the workload of each node could enhance performance by redistributing tasks. By monitoring the computational load and adjusting the assignment of grid segments accordingly, we can ensure a more balanced use of resources when the distribution of live cells is uneven. This approach would improve overall efficiency and reduce the time to complete each simulation turn.

VI. DISTRIBUTED IMPLEMENTATION - CONCLUSION

The distributed Game implementation of Life enhances computational efficiency and scalability by utilizing multiple AWS nodes and optimizing communication protocols. The introduction of the Halo Exchange mechanism significantly reduces communication overhead, resulting in notable performance improvements, especially on larger grids. The extension with SDL Live View had a minimal impact on performance, indicating communication bottlenecks were effectively addressed through Halo Exchange.

However, despite these optimizations, system performance is still influenced by load imbalances across worker nodes, particularly when dealing with uneven cell distributions. Introducing dynamic load balancing could further improve system efficiency. In addition, while real-time SDL updates provided valuable visualization, their minimal impact on performance highlights the system's robustness under current optimizations.